



YILDIZ TECHNICAL UNIVERSITY
MECHATRONICS ENGINEERING
MACHINE LEARNING – MKT3434
HW-03

Name and Surname: Enis Büyükbayrak

Student ID Number: 21067004

1. Advanced Deep Learning Enhancements

- Derin öğrenme sekmesi, yapılan son geliştirmelerle birlikte artık yalnızca temel MLP yapılarıyla sınırlı kalmamakta, aynı zamanda görsel ve zamana bağlı veri setleri üzerinde gelişmiş mimariler oluşturulmasına olanak tanımaktadır. Bu kapsamda kullanıcılar, GUI arayüzü üzerinden sezgisel olarak karmaşık ağlar inşa edebilir, eğitim sürecini takip edebilir ve modellerini dışa aktararak farklı projelerde kullanabilir.

a) Model Configuration and Layer Management

- Katman Ekleme Desteği: Kullanıcılar, GUI arayüzü üzerinden çok sayıda katman ekleyerek esnek yapay sinir ağı mimarileri oluşturabilir. Her katmanın nöron sayısı ve aktivasyon fonksiyonu (ReLU, Sigmoid, Tanh) kullanıcı tarafından belirlenebilir.
- Görüntü İşleme için Yapılar: Görsel veriler için özel olarak Conv2D (evrimsel) ve MaxPooling2D (havuzlama) katmanları desteklenmektedir. Kullanıcı, filtre sayısı, kernel boyutu gibi parametreleri ayarlayarak görüntüdeki mekansal özellikleri yakalayabilir.
- Zamansal Veriler için RNN Desteği: Sekme üzerinden LSTM (Long Short-Term Memory) ve GRU (Gated Recurrent Unit) katmanları eklenerek zaman serileri gibi sıralı veriler işlenebilir. Bu, özellikle sinyaller, metin veya zaman tabanlı olaylar için büyük avantaj sağlar.
- Katmanları Kaydetme/Yükleme: Oluşturulan modeller, .h5 ve .json formatlarında kaydedilebilir ve tekrar GUI'ye yüklenerek eğitim devam ettirilebilir veya dağıtımına hazırlanabilir.
- Katmanları Dinamik Olarak Silme/Ekleme: Arayüz üzerinden kullanıcı, oluşturduğu katmanları kolayca düzenleyebilir, silebilir veya yeni katmanlar ekleyerek yapıyı güncelleyebilir.

b) Training and Optimization

- Optimizasyon Seçenekleri: Eğitim sürecinde farklı optimizasyon algoritmaları (Adam, SGD, RMSprop) kullanılabilir.
- Öğrenme Hızı Zamanlaması: Step Decay ve Exponential Decay gibi öğrenme oranı zamanlamaları GUI üzerinden ayarlanabilir, böylece eğitim sürecinde öğrenme oranı dinamik olarak değiştirilebilir.
- Düzenleme (Regularization) Teknikleri: Overfitting'in önüne geçmek için Dropout ve L2 regularizasyonu desteklenmektedir.
- Doğrulama Kayıp İzleme: Eğitim sırasında doğrulama verisi üzerinden kayıp takibi yapılır ve overfitting erken tespit edilerek müdahale edilebilir.

c) Training Visualization and Evaluation

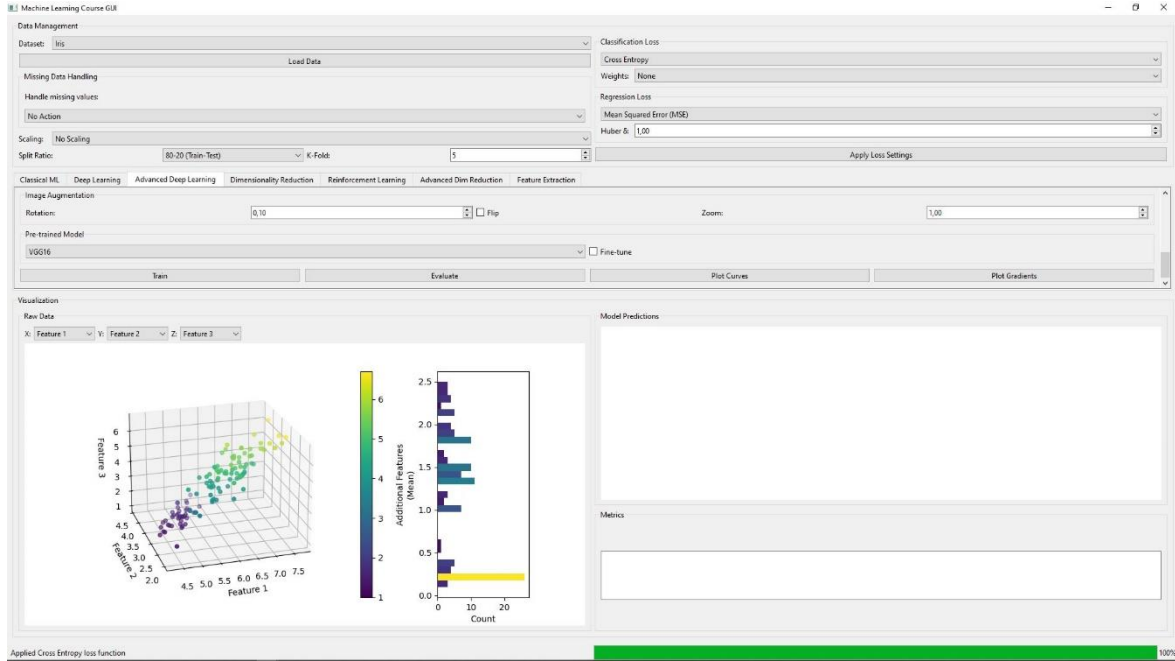
- Eğitim Geçmişi Görselleştirme: Model eğitildikçe epoch bazlı olarak loss ve accuracy değerleri matplotlib kullanılarak canlı olarak grafik şeklinde sunulur.

- Test Sonuçları: Eğitim tamamlandıktan sonra test verisi üzerinde accuracy ve score gibi önemli metrikler hesaplanarak kullanıcıya sunulur.
- Confusion Matrix ve Karşılaştırmalar: Sınıflandırma problemleri için tahminler ile gerçek etiketler karşılaştırılarak karışıklık matrisi görüntülenir.

d) Transfer Learning and Data Augmentation

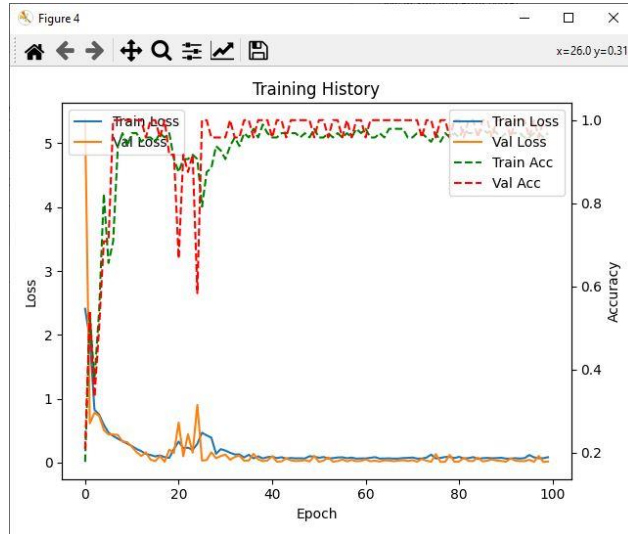
- Transfer Öğrenme: Kullanıcılar önceden eğitilmiş modelleri (VGG16, ResNet gibi) GUI üzerinden yükleyip, kendi özel veri setleri üzerinde fine-tune edebilir.
- Veri Artırımı (Data Augmentation): Görsel veri setleri için dönme (rotation), yansıtma (flipping) ve ölçekleme (scaling) gibi artırma yöntemleri uygulanabilir. Bu sayede modelin genelleme kapasitesi artırılır ve veri seti çeşitliliği zenginleştirilir.
- Model Mimarisi ve Seçim Gerekçeleri
- Aşağıda kullanılan tipik bir CNN mimarisinin katman yapısı ve bu yapıların tercih edilme nedenleri sunulmuştur:
- Conv2D Katmanları: Görsel özniteliklerin çıkarılması için kullanılır. 32, 64 ve 128 filtrelik katmanlar sayesinde model hem genel hem de detaylı özellikleri yakalayabilir. Kernel boyutu olarak genellikle (3x3) kullanılmıştır; bu boyut, işlem yükünü artırmadan lokal yapıları etkili biçimde tespit etmeye olanak sağlar.
- MaxPooling2D Katmanları: (2x2) havuzlama işlemi, boyutları azaltarak işlem maliyetini düşürürken aynı zamanda aşırı öğrenmeyi (overfitting) engeller.
- Dropout Katmanları: Özellikle dense katmanlardan sonra yerleştirilen %50 oranlı dropout, her epoch'ta bazı nöronların rastgele devre dışı bırakılmasını sağlayarak modelin daha dayanıklı olmasına katkıda bulunur.
- Flatten Katmanı: 2D feature map'leri 1D vektöre dönüştürerek dense katmanlar için uygun giriş oluşturur.
- Dense Katmanlar: Girişten çıkarılan özellikleri işleyerek nihai sınıflandırmayı sağlar. Softmax aktivasyonu sayesinde çıkışta her sınıf için olasılık değeri üretilir.
- Özellik Çıkarma Verimliliği: Çok katmanlı Conv2D ve MaxPooling yapıları, verinin farklı seviyedeki yapısal özelliklerini hiyerarşik olarak öğrenmeyi mümkün kılar.
- Genelleme Kabiliyeti: Dropout gibi regularizasyon teknikleri ile küçük veri setlerinde bile genelleme başarısı artırılır.
- Denge: Sunulan yapı, model karmaşıklığı, eğitim süresi ve doğruluk oranı açısından iyi bir denge sunar. Bu yapı, genel amaçlı görüntü sınıflandırma problemleri için oldukça uygundur.

2. SCREENSHOTS



1. New GUI

- Burada Deep ve Advanced Deep Learning tablaları işlevsel hale getirilmiştir. Belirli parametrelerin değiştirilmesi ve layer ekleme özellikleri eklenmiştir.



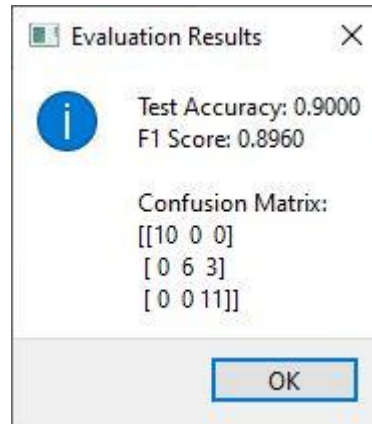
2. Training Output

- Burada train edilen bir modelin istenen parameter doğrultusunda tek plot üzerinde gösterilmesi görülmektedir.

```
3/3 ————— 0s 13ms/step - accuracy: 0.9727 - loss: 0.0572 - val_accuracy: 1.0000 - val_loss: 0.0194
Epoch 89/100
3/3 ————— 0s 13ms/step - accuracy: 0.9557 - loss: 0.0705 - val_accuracy: 0.9583 - val_loss: 0.1273
Epoch 90/100
3/3 ————— 0s 12ms/step - accuracy: 0.9596 - loss: 0.1017 - val_accuracy: 1.0000 - val_loss: 0.0162
Epoch 91/100
3/3 ————— 0s 12ms/step - accuracy: 0.9635 - loss: 0.0649 - val_accuracy: 1.0000 - val_loss: 0.0312
Epoch 92/100
3/3 ————— 0s 13ms/step - accuracy: 0.9674 - loss: 0.0623 - val_accuracy: 1.0000 - val_loss: 0.0326
Epoch 93/100
3/3 ————— 0s 12ms/step - accuracy: 0.9805 - loss: 0.0784 - val_accuracy: 1.0000 - val_loss: 0.0127
Epoch 94/100
3/3 ————— 0s 14ms/step - accuracy: 0.9049 - loss: 0.1900 - val_accuracy: 1.0000 - val_loss: 0.0233
Epoch 95/100
3/3 ————— 0s 13ms/step - accuracy: 0.9492 - loss: 0.1403 - val_accuracy: 1.0000 - val_loss: 0.0218
Epoch 96/100
3/3 ————— 0s 12ms/step - accuracy: 0.9414 - loss: 0.1243 - val_accuracy: 0.9167 - val_loss: 0.2179
Epoch 97/100
3/3 ————— 0s 13ms/step - accuracy: 0.9779 - loss: 0.0907 - val_accuracy: 1.0000 - val_loss: 0.0384
Epoch 98/100
3/3 ————— 0s 12ms/step - accuracy: 0.9440 - loss: 0.1565 - val_accuracy: 1.0000 - val_loss: 0.0599
Epoch 99/100
3/3 ————— 0s 15ms/step - accuracy: 0.9414 - loss: 0.1273 - val_accuracy: 1.0000 - val_loss: 0.0578
Epoch 100/100
3/3 ————— 0s 12ms/step - accuracy: 0.9518 - loss: 0.0887 - val_accuracy: 1.0000 - val_loss: 0.0221
QCoreApplication::exec: The event loop is already running
```

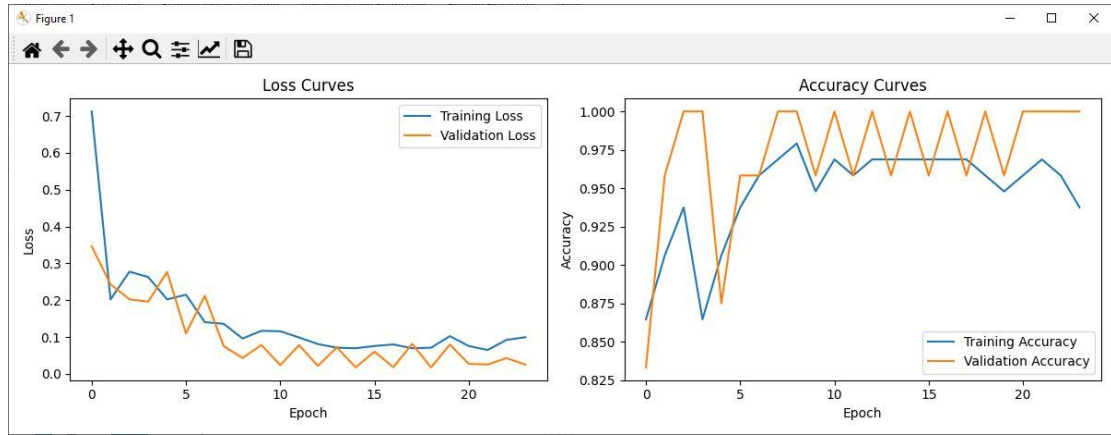
3. Terminal Output

- Terminal çıktısıyla süregelen işlemin epoch ,geri kalan loss ve accuracy değerleri net bir şekilde çıktı olarak alınmaktadır ek olarak bunların log işlemleri halen yapılmaya çalışılmaktadır.



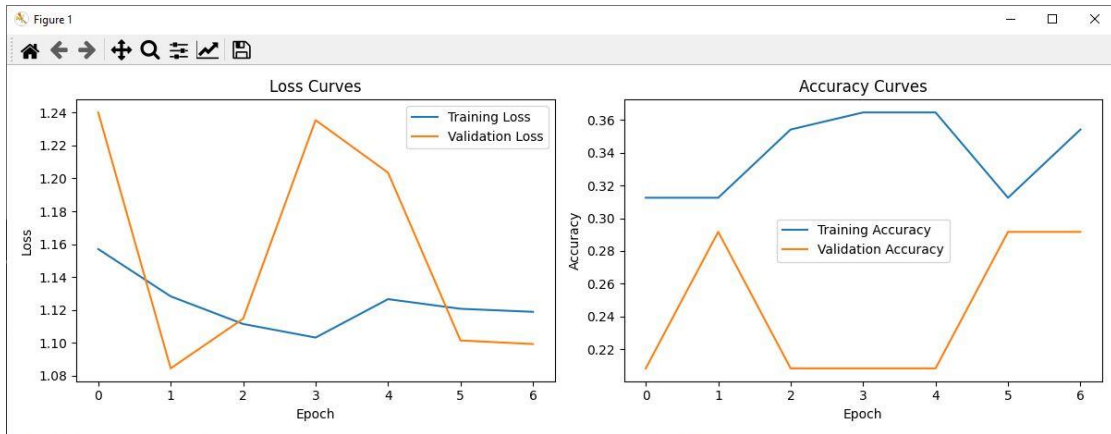
4. Evaluate Output

- Terminal çıktısıyla süregelen işlemin epoch ,geri kalan loss ve accuracy değerleri net bir şekilde çıktı olarak alınmaktadır ek olarak bunların log işlemleri halen yapılmaya çalışılmaktadır.



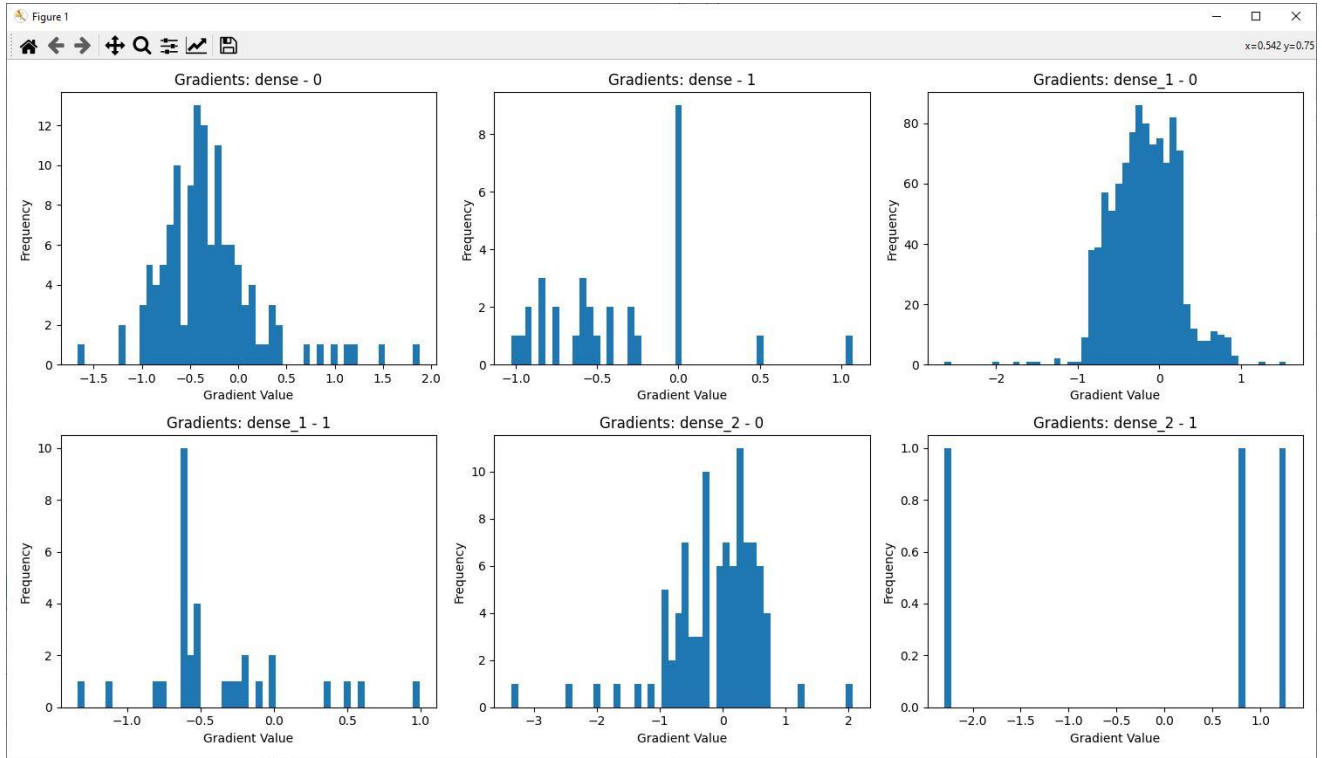
5. ADAM with RestNET50 Output

- Adam ile oluşturan model çıktısı bu şekildedir fine tuning uygulanmıştır.



6. SGD with RestNET50 Output

- SGD ile oluşturan model çıktısı bu şekildedir fine tuning uygulanmıştır.



7. Gradient Out

- Train edilen modelin Gradient Plotlarıdır her bir aralık için tek tek hesaplanıp tek yerde gösterilmiştir.

```

1 import os
2 os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
3 import sys
4 import numpy as np
5 import pandas as pd
6 from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
7                               QHBoxLayout, QLabel, QPushButton, QFileDialog,
8                               QComboBox, QSpinBox, QDoubleSpinBox,
9                               QGroupBox, QScrollArea, QTextEdit, QStatusBar,
10                              QProgressBar, QCheckBox, QGridLayout, QMessageBox,
11                              QDialog, QLineEdit, QListWidget, QListWidgetItem)
12 from PyQt6.QtCore import Qt, QThread, pyqtSignal
13 import matplotlib.pyplot as plt
14 from matplotlib.backends.backend_qt5agg import FigureCanvasQTAgg as FigureCanvas
15 from matplotlib.figure import Figure
16 from sklearn import datasets, preprocessing, model_selection
17 from sklearn.linear_model import LinearRegression, LogisticRegression
18 from sklearn.naive_bayes import GaussianNB
19 from sklearn.svm import SVC
20 from sklearn.tree import DecisionTreeClassifier
21 from sklearn.ensemble import RandomForestClassifier
22 from sklearn.neighbors import KNeighborsClassifier
23 from sklearn.cluster import KMeans
24 from sklearn.decomposition import PCA
25 from sklearn.metrics import accuracy_score, mean_squared_error, confusion_matrix
26 from sklearn.impute import SimpleImputer
27 from sklearn.decomposition import PCA
28 from sklearn.decomposition import TruncatedSVD
29 from sklearn.manifold import TSNE
30 from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
31 from sklearn.metrics import silhouette_score
32 from sklearn.metrics import pairwise_distances
33 import tensorflow as tf
34 from tensorflow.keras import layers, models, optimizers
35 import logging
36 from datetime import datetime

```

8. Real-Time Logging

- GUI üzerinde yapılan tüm işlemler bunların yanında ortaya çıkan hata mesajları ve her model eğitiminde süre gelen bütün işlemler anlık olarak log kayıtlarında tutulabilmektedir. Yapılması planlanan yeni güncellemede ise logların hepsi belirli bir dosya içine aktarılması ve öncelik sırası yapılması amaçlanmaktadır.